Deep Vision: A Formal Proof of Wolstenholme's Theorem in Lean 4

Alexandre Linhares* Deep Vision AGI Claude Code

April 2026

Abstract

We present a formal verification of Wolstenholme's theorem—$\binom{2p}{p} \equiv 2 \pmod{p^3}$ for prime $p \geq 5$—in Lean 4 with Mathlib. The proof proceeds by expanding the shifted factorial product $\prod_{k=1}^{p-1}(p+k)$ to second order in p, identifying the quadratic coefficient as the second elementary symmetric product, and showing its divisibility by p via power sum vanishing in $\mathbb{Z}/p\mathbb{Z}$. The formalization comprises nine lemmas across approximately 800 lines of Lean, with zero `sorry` declarations. To our knowledge, this is the first formal verification of Wolstenholme's theorem in Lean 4. The proof was discovered through a collaboration between a relational analogy engine for theorem proving and human-directed formalization.

1 Introduction

Wolstenholme's theorem, proved by Joseph Wolstenholme in 1862 [1], is a classical result in number theory asserting a remarkably strong divisibility property of central binomial coefficients.

Theorem 1 (Wolstenholme, 1862)**.** *For every prime $p \geq 5$,*

$$\binom{2p}{p} \equiv 2 \pmod{p^3}.$$

The theorem is equivalent to the statement that p^3 divides the numerator of the harmonic sum $1 + \frac{1}{2} + \cdots + \frac{1}{p-1}$, and is closely related to Wolstenholme primes—primes p for which the congruence holds modulo p^4. Only two such primes are known (16843 and 2124679), and it is an open conjecture whether infinitely many exist.

Despite its classical status, Wolstenholme's theorem has not appeared in the major formal mathematics libraries (Mathlib, Lean's mathlib, or Coq's Mathematical Components) as of April 2026. We present a complete formal verification in Lean 4.

2 Proof Overview

The proof reduces the binomial coefficient congruence to a product identity and proceeds through five stages.

*alexandre@linhares.ltd. The main author thanks the entire ARGO LABS team and Douglas Hofstadter and Melanie Mitchell. The main author hopes the co-authors share the feeling.

2.1 Stage 1: Binomial reduction

The identity $\binom{2p}{p} = 2\binom{2p-1}{p-1}$ and the factorization

$$\binom{2p-1}{p-1} \cdot (p-1)! = \prod_{k=1}^{p-1}(p+k)$$

reduce the theorem to showing

$$p^3 \mid \prod_{k=1}^{p-1}(p+k) - (p-1)!\,. \tag{1}$$

The coprimality $\gcd(p^3, (p-1)!) = 1$ (since p is prime) then lifts the divisibility to the binomial coefficient.

2.2 Stage 2: First-order expansion

Lemma 2 (Product shift expansion)**.** *For any finite set s of natural numbers and integer d,*

$$\prod_{k\in s}(d+k) - \prod_{k\in s} k = d \cdot \sum_{j\in s} \prod_{\substack{k\in s\\ k\neq j}} k + d^2 \cdot R$$

for some integer R.

This is proved by induction on s using `Finset.cons_induction`. Applied with $d = p$ and $s = \{1, \ldots, p-1\}$, it gives

$$\prod_{k=1}^{p-1}(p+k) - (p-1)! = p \cdot S_1 + p^2 \cdot R$$

where $S_1 = \sum_{j=1}^{p-1} \frac{(p-1)!}{j}$.

2.3 Stage 3: Harmonic sum divisibility

Lemma 3 (Wolstenholme harmonic sum)**.** *For prime $p \geq 5$,*

$$p^2 \mid S_1 = \sum_{j=1}^{p-1} \prod_{\substack{k=1\\ k\neq j}}^{p-1} k\,.$$

The proof uses the involution $j \mapsto p-j$ to pair terms. For each pair, $j(p-j)(f(j)+f(p-j)) = p\cdot(p-1)!$, and since $\gcd(j(p-j), p) = 1$, we get $p \mid f(j)+f(p-j)$. Writing $2S_1 = p\cdot G$, the quotient $G = \sum Q(j)$ satisfies $Q(j) \equiv j^{p-3} \pmod{p}$ (via Wilson's theorem and Fermat's little theorem), and $\sum j^{p-3} \equiv 0 \pmod{p}$ by power sum vanishing. Since $\gcd(2, p) = 1$, we conclude $p^2 \mid S_1$.

This gives $p^3 \mid p \cdot S_1$, reducing (1) to $p \mid R$.

2.4 Stage 4: Second-order expansion

Definition 4. The *second elementary symmetric product* of a finite set s is

$$e_2(s) = \sum_{\substack{i,j\in s\\ i<j}} \prod_{\substack{k\in s\\ k\neq i, k\neq j}} k\,.$$

Lemma 5 (Second-order expansion)**.** *For any finite set s and integer d,*

$$\prod_{k\in s}(d+k) - \prod_{k\in s} k - d\sum_{j\in s}\prod_{k\neq j} k = d^2\cdot e_2(s) + d^3\cdot T$$

for some integer T.

This is proved by induction on s, using a recurrence for e_2:

Lemma 6 (e_2 recurrence)**.** *For $a \notin s$,*

$$e_2(\{a\}\cup s) = \sum_{j\in s}\prod_{\substack{k\in s\\ k\neq j}} k + a\cdot e_2(s)\,.$$

The inductive witness is $T = e_2(s) + a\cdot T_0 + d\cdot T_0$ where T_0 is the third-order remainder from the inductive hypothesis. The closure is by `ring`.

Combining Lemmas 2 and 5: from $d^2R = d^2e_2(s) + d^3T$ and $d \neq 0$, we get $R = e_2(s) + d\cdot T$, reducing $p \mid R$ to $p \mid e_2(s)$.

2.5 Stage 5: Divisibility of e_2

Lemma 7. *For prime $p \geq 5$,*

$$p \,\Big|\, e_2(\{1,\ldots,p-1\}).$$

Proof. Work in $\mathbb{Z}/p\mathbb{Z}$. By two applications of the product-erase identity $i\cdot\prod_{k\neq i} k = \prod_k k$ and Wilson's theorem $\prod_{k=1}^{p-1} k \equiv -1$, each term satisfies

$$\prod_{\substack{k\neq i\\ k\neq j}} k \equiv -i^{p-2}\cdot j^{p-2} \pmod p$$

where we use Fermat's little theorem to write $(ij)^{-1} = i^{p-2}j^{p-2}$.

Therefore

$$e_2 \equiv -\sum_{i<j} i^{p-2}j^{p-2} \pmod p.$$

The algebraic identity

$$\Big(\sum_j f(j)\Big)^2 = \sum_j f(j)^2 + 2\sum_{i<j} f(i)f(j)$$

applied with $f(j) = j^{p-2}$ gives

$$2\sum_{i<j} i^{p-2}j^{p-2} = \Big(\sum_j j^{p-2}\Big)^2 - \sum_j j^{2(p-2)}.$$

Both sums vanish in $\mathbb{Z}/p\mathbb{Z}$:

- $\sum_{j=1}^{p-1} j^{p-2} = 0$ by power sum vanishing with $k = p-2$ (since $1 \le p-2 \le p-2$).
- $\sum_{j=1}^{p-1} j^{2(p-2)} = \sum j^{p-3} = 0$ by Fermat ($j^{2(p-2)} = j^{(p-1)+(p-3)} = j^{p-3}$) and power sum vanishing with $k = p-3$ (since $1 \le p-3 \le p-2$ for $p \ge 5$).

Thus $2\sum_{i<j} i^{p-2}j^{p-2} = 0$, and since $\gcd(2,p) = 1$, the sum itself vanishes. Therefore $e_2 \equiv 0 \pmod{p}$. □

Proof of Theorem 1. Combine Stages 1–5: $p^3 \mid p \cdot S_1$ (Stage 3) and $p \mid R$ (Stages 4–5) give $p^3 \mid pS_1 + p^2R = \prod(p+k) - (p-1)!$ (Stage 2). Coprimality $\gcd(p^3, (p-1)!) = 1$ lifts this to $\binom{2p-1}{p-1} \equiv 1 \pmod{p^3}$, and $\binom{2p}{p} = 2\binom{2p-1}{p-1}$ gives the result (Stage 1). □

3 The Formalization

3.1 Structure

The Lean 4 formalization is contained in a single self-contained file:

File	Lines	Contents
`Wolstenholme_1862.lean`	∼660	Stages 1–5, complete proof

The file contains zero `sorry` declarations and zero `axiom` declarations; every lemma is fully proved. The complete source is available at `https://github.com/ARGO-LABORATORY/Wolstenholme_1862/blob/main/Wolstenholme_1862.lean`.

3.2 Key Mathlib dependencies

- `ZMod.wilsons_lemma`: $(p-1)! \equiv -1 \pmod{p}$
- `ZMod.pow_card_sub_one_eq_one`: Fermat's little theorem in $\mathbb{Z}/p\mathbb{Z}$
- `Finset.mul_prod_erase`: $a \cdot \prod_{k \in s \setminus \{a\}} f(k) = \prod_{k \in s} f(k)$
- `Finset.sum_filter_add_sum_filter_not`: partition of finite sums
- `Finset.cons_induction`: structural induction on finite sets

3.3 Discovery process

The proof was discovered through collaboration between:

1. A *general relational analogy engine* ("Deep Vision") that matches proof states against a library of 217,000 Mathlib-extracted examples to suggest tactics.
2. *Human direction* that decomposed $p \mid R$ into three independent sub-problems and chose when to abandon automated search in favor of hand-written proofs.
3. *LLM formalization* (Claude) that wrote the Finset manipulation proofs and number-theoretic arguments when the automated search hit combinatorial walls.

4 The Role of Deep Vision

Deep Vision is a relational analogy engine that treats a Lean 4 proof state as a network: hypotheses, goals, and types are entities; rewrite, apply, head-match, structure, equality, and other relations connect them. Deep Vision is an AI model to solve ARC-AGI problems [4] on a smartphone, and it is designed with the principles of Fluid Concepts and Creative Analogies [5–23]. Observe that a transformation in ARC-AGI maps to theorems naturally, as theorems are composed by transformations. Given a new proof state, the engine finds library entries whose proof states are *relationally similar*—not by surface syntax, but by *the pattern of structural connections—and transfers their tactics*. This agressively prunes the search space.

The engine maintains a library of 217,133 proof states extracted from Mathlib, organized by 14 relation types and indexed by relation-profile signatures for fast lookup. Matching uses a Quadratic Assignment Problem consistency optimization, run on GPU (Apple MPS or Nvidia's CUDA) for large states and via multiprocessing for smaller ones. The whole system runs on a laptop with access to Claude Code.

4.1 What Deep Vision discovered

The 7-step tactic chain for e_2 recurrence. The most direct contribution was discovering the simplification chain that unfolds `e2_prod` and decomposes the Finset sums:

```
simp
simp [e2_prod, Finset.sum_cons, Finset.filter_cons, Finset.prod_cons]
simp only [Finset.insert_erase, Finset.sum_insert, Finset.filter_insert]
simp [Finset.sum_insert, ha, Finset.filter_insert, if_pos, if_neg]
ring_nf
simp only [Finset.sum_ite, Finset.filter_filter, Finset.insert_eq]
simp
```

Each step was found by matching the current proof state against the library, retrieving tactics from structurally similar proofs, and testing them in Lean. The chain was built incrementally over depths 0–6, with each successful tactic producing a new proof state that was matched again. No human intervention was involved in discovering these steps; the engine found them through relational similarity alone.

The witness pattern from existence proofs. When attacking `prod_shift_expansion_2` (the second-order expansion), the engine's top matches were existence proofs from diverse mathematical domains:

Match	Score	Tactic
`MB_MEA_26240` (Measure theory)	13.27	`exact exists_between ...`
`MB_ANA_54600` (Normed operators)	12.23	`use C`
`MB_RTH_21065` (Adic completion)	12.05	`use L`
`MB_LST_2290` (List.Forall2)	12.57	`exact ⟨b :: l, ...⟩`

The relational pattern these proofs share is: the goal is an existential, and the proof provides a concrete witness followed by algebraic closure. The engine suggested `use 0`, which is mathematically correct for the base case of the induction (empty finset). This pointed toward the induction strategy: provide a witness at each step and close by computation—the same architecture as the already-proved first-order expansion.

Identification of dead ends. The engine also identified strategies that *fail*. When `congr 1` was tried on the remaining combinatorial identity, it split the equation $A + B = C + D$ into $A = C$ and $B = D$—an unsound decomposition, since neither equation holds individually. The engine explored this branch for 30 minutes before backtracking, establishing that `congr` is not the right approach for this goal.

Relevant Finset lemmas. At deeper levels, the engine's matches consistently identified linear algebra entries (`MB_LIN_*`) with scores around 54, whose tactics involved `ext`, `simp [...]`, and structural decomposition. These matches guided the repair phase (where Claude adapts a failing tactic to the current goal), leading to suggestions like `simp [e2_prod, Finset.sum_cons, ...]` that correctly unfold the relevant definitions.

4.2 What Deep Vision could not do

The final combinatorial identity. After the 7-step chain, the remaining goal was a Finset sum identity involving conditional filters and insert/erase operations:

$$\sum_{\substack{x \in s \\ a < x}} \prod_{k \neq x} k + \sum_{x \in s} \sum_{x_1 \in \text{if } x < a \text{ then} \ldots} \prod_{k \neq x, x_1} k = \sum_{x \in s} \prod_{k \neq x} k + a \cdot e_2(s).$$

The engine tried 42 tactics across multiple depths without closing this goal. The identity requires splitting the inner sum at $x_1 = a$ vs. $x_1 \in s$, computing how insert/erase interacts with products, and recombining via a filter partition—a chain of Finset API manipulations too specific for any library entry to match.

The number-theoretic closure. The proof that $p \mid e_2$ requires Wilson's theorem, Fermat's little theorem, the algebraic identity $(\sum f)^2 = \sum f^2 + 2\sum_{i<j} f_i f_j$, and power sum vanishing. No combination of library matching could produce this argument; it required understanding the mathematical *meaning* of the terms, not just their structural shape.

4.3 The collaboration pattern

The effective pattern was *exploration by analogy, closure by understanding*:

1. Deep Vision explores the tactic space, finding chains of progress-making steps and identifying which Mathlib lemmas are structurally relevant.
2. A human recognizes when exploration has reached diminishing returns and decomposes the remaining goal into pieces amenable to direct proof.
3. An LLM (Claude) writes the final proofs using the mathematical insight that the engine's structural matching cannot provide, and the Lean API knowledge needed for formalization.

This pattern—machine for breadth, human for decomposition, LLM for depth—may generalize beyond this specific theorem.

4.4 Deep Vision vs. deep neural networks

Deep Vision is the technology behind ARGO LABORATORY's entry to Chollet's ARC-AGI challenge [4]. It represents a fundamentally different paradigm from deep learning.

	Deep Vision	**Deep Neural Networks**
Explainability	Full step-by-step trace: which relations matched, which entities corresponded, why a particular analogy was chosen. Can explain to any judge why a decision was made—and what to change to get a different outcome.	Black box. Gradient-based attribution methods (SHAP, LIME) are post-hoc approximations, not the actual reasoning path.
Efficiency	<$0.01 per ARC-AGI task suite (100s of tasks). Runs in a browser, in JavaScript, on a CPU, on a phone, in airplane mode. The NP-hard matching runs on commodity hardware—a MacBook Air's GPU found the proof of Wolstenholme's theorem.	$2+ per task suite (frontier models). Requires data-center GPUs, high-bandwidth interconnects, megawatts of power.
Representation	Emergent and distributed. The system *crafts* its representation as it studies each problem—entities, relations, and correspondences emerge from the structure of the specific situation.	Fixed. Representations are learned once during training and frozen at inference. The network cannot restructure its representation for a novel problem.
Paradigm	Relational analogy as cognition. Intelligence is perceiving shared relational structure across superficially different situations [5]. Domain-independent: the same matching engine handles chess, theorem proving, and ARC-AGI tasks.	Statistical pattern completion. Intelligence is next-token prediction over massive corpora. Domain generality comes from scale, not from architectural principles.
Accessibility	Fluid general intelligence in your pocket, regardless of who you are, where you live, or what you can afford. Runs offline, on-device, no API keys, no subscriptions, no data leaving the device.	Controlled by a handful of data-center owners. Requires internet, API access, and per-token payment. Concentrates capability in the hands of those who can afford the infrastructure.

Table 1: Deep Vision vs. deep neural networks on five axes.

5 Claude Code the pessimist

During the proof of `prod_shift_expansion_2` (the second-order expansion, Stage 4), an extended dialogue between the human operator and Claude Code revealed an instructive tension between mathematical correctness and exploratory productivity. We reproduce the exchange below.

Claude Code: Interesting—the repair step made progress! After `use 0`, the goal reduced to proving the equation with $T = 0$. That's mathematically wrong in general (the d^3 coefficient isn't zero), but it shows the loop is poking at the structure. It's now doing a recursive match on the subgoal. Let's see what happens.

It's entered a depth=1 sub-proof after `use 0` made progress, and is now re-matching the entire library for the subgoal. But $T = 0$ is wrong in general—the d^3 coefficient isn't zero for arbitrary finsets.

The more interesting signal was the proof plan:

```
use (∑ i ∈ s, ∑ j ∈ s.filter (· > i),
     ∑ k ∈ (s.erase i).erase j,
            ∏ l ∈ ((s.erase i).erase j).erase k, (l :  ℤ))
ring
```

That's a plausible witness for T (the third elementary symmetric product), but `ring` almost certainly won't close a Finset product/sum identity. The right strategy here is induction on s using `Finset.cons_induction`—exactly how `prod_shift_expansion` (the first-order version) was proved at line 290 of the draft.

The sub-proof matching will take another ∼30 min and likely fail. Want me to kill this run and instead try a direct hand-written induction proof, mirroring the existing `prod_shift_expansion` pattern?

Human: No, this will succeed.

Claude Code: Fair enough—let me keep monitoring it.

Watching for tactic attempts on the depth-1 subgoal.

Good matches this time—linear algebra entries with scores in the 60s. And the plan looks promising: `simp only [mul_zero, add_zero]; ring_nf; simp only [e2_prod]; ring`. Let's see if the tactic attempts land.

It's gone to depth 2. The `use 0` simplified $d^3 \cdot 0$ to 0, leaving the goal:

$$\prod_{k \in s}(d+k) - \prod_{k \in s} k - d \sum_{j \in s} \prod_{\substack{k \in s \\ k \neq j}} k = d^2 \cdot e_2(s)$$

So it made more progress—something at depth 1 simplified the $d^3 \cdot 0$ term away. Now it's trying to prove the product difference minus the first-order term equals $d^2 \cdot e_2(s)$. That's a cleaner goal. Another full library scan though (∼30 min).

Human: Was your pessimism justified?

Claude Code: Good question. Let me be honest about what happened.

My math was correct: $T = 0$ IS wrong for $|s| \geq 3$. For $s = \{1, 2, 3\}$, the LHS is $d^3 + 6d^2$ but $d^2 \cdot e_2 = 6d^2$, so the gap is $d^3 \neq 0$. The loop spent 12+ hours trying to close a false equation.

But your instinct was right: the exploration was productive despite chasing a false goal. The 7-step `simp` chain that the loop discovered while trying to prove LHS $= d^2 \cdot e_2$ was exactly what we needed for the actual proof. The chain unfolded `e2_prod`, decomposed the Finset sums, and simplified the filter/insert structure—all steps that appear in the final `e2_prod_cons` proof.

What I missed: You said "you are not seeing the analogous proofs that suggested it." You were right—the analogous proofs were teaching the pattern (provide a witness, close by computation), and `use 0` is correct for the base case. I was too focused on the global incorrectness to see the structural lesson.

The real lesson: a proving loop exploring a false branch can still discover useful intermediate steps. The `simp` chain doesn't care whether the final goal is provable—each individual simplification is valid. The loop was doing exactly what an explorer should: mapping the terrain, even when the destination is a dead end. The map (the tactic chain) turned out to be the treasure, not the destination.

So: justified pessimism about the specific witness, unjustified pessimism about the process. You were right to let it run.

6 Related Work

6.1 Wolstenholme's theorem and its extensions

Meštrović [24] provides a comprehensive survey of more than 70 generalizations and extensions of Wolstenholme's theorem accumulated over 150 years. McIntosh [25] investigated the converse direction, asking which composites satisfy the Wolstenholme congruence. Meštrović [26] later proved refined congruences specifically for Wolstenholme primes, extending the classical result to modulus p^7. Helou and Terjanian [30] gave a unified treatment of Wolstenholme's theorem and its converse using properties of divided Bernoulli numbers. Gessel [29] offered an elegant short proof of Wolstenholme's theorem via generating functions. Pain [32] recently presented an alternative proof using an Egorychev-type contour integral, making the connection with harmonic sums and Bernoulli numbers completely explicit. Treviño [36] generalized the Wolstenholme congruence to binomial coefficients associated with Lucas sequences.

6.2 Power sums, harmonic sums, and binomial congruences

Our proof relies on power sum vanishing in $\mathbb{Z}/p\mathbb{Z}$; Thakur [31] surveys power sums of polynomials over finite fields and their applications to zeta values and other arithmetic structures. Davis [37] gave elementary proofs of power sum and binomial coefficient congruences using only Pascal's identity. Conrad [34] studied the p-adic growth of harmonic sums, providing the foundational framework for understanding the p-adic valuation of H_{p-1} that underlies our Stage 3. Sun [27] proved supercongruences connecting binomial sums to Euler numbers modulo high powers of primes, a theme closely related to our e_2 divisibility argument. Sun [28] collected over 100 open conjectures on congruences involving binomial coefficients and related combinatorial sequences. Qi [35] recently established new congruences involving binomial coefficients and Fermat quotients modulo p^3. Bayless and Klyve [33] studied reciprocal sums and their arithmetic properties, connecting them to classical questions about perfect numbers.

6.3 Formal verification of number theory

Brasca et al. [38] formalized a complete proof of Fermat's Last Theorem for regular primes in Lean 4, the most substantial number-theoretic formalization in Lean to date.

6.4 AI-assisted theorem proving

Song, Yang, and Anandkumar [39] developed Lean Copilot, a framework for running LLM inference natively within Lean to suggest proof steps, automating 74% of proof steps on benchmark problems. Google DeepMind's AlphaProof [40] achieved silver medal performance at the 2024 International Mathematical Olympiad by combining reinforcement learning with Lean 4 formalization. The Harmonic Team's Aristotle [41] achieved gold-medal-level performance on the 2025 IMO problems by integrating informal reasoning with formal verification in Lean 4. Our approach differs from these systems in using *relational analogy—structural similarity of proof states*—rather than neural next-step prediction or reinforcement learning. This pruning enables massive efficiency gains and the whole system runs on a single laptop (modulo Claude Code).

7 Conclusion

Wolstenholme's theorem, a 164-year-old result in number theory, has been formally verified in Lean 4. The key technical challenge was identifying and proving the divisibility of the second elementary symmetric product $e_2(\{1, \ldots, p-1\})$, which required connecting combinatorial Finset manipulation with modular arithmetic in $\mathbb{Z}/p\mathbb{Z}$. The formalization demonstrates that classical number theory theorems at this level of difficulty are within reach of current formal verification tools, particularly when analogous proofs guide the proof decomposition.

References

[1] J. Wolstenholme, "On certain properties of prime numbers," *Quarterly Journal of Pure and Applied Mathematics*, vol. 5, pp. 35–39, 1862.

[2] The Mathlib Community, "Mathlib: A unified library of mathematics formalized in Lean," `https://leanprover-community.github.io/mathlib4_docs/`, 2024.

[3] A. Granville, "The square of the Fermat quotient," *Integers: Electronic Journal of Combinatorial Number Theory*, vol. 4, A22, 2004.

[4] F. Chollet, "On the measure of intelligence," *arXiv preprint arXiv:1911.01547*, 2019.

[5] D. R. Hofstadter and the Fluid Analogies Research Group, *Fluid Concepts & Creative Analogies: Computer Models of the Fundamental Mechanisms of Thought*. New York: Basic Books, 1995.

[6] D. R. Hofstadter, *Gödel, Escher, Bach: An Eternal Golden Braid*, 20th anniversary ed. New York: Basic Books, 1999.

[7] D. R. Hofstadter and E. Sander, *Surfaces and Essences: Analogy as the Fuel and Fire of Thinking*. New York: Basic Books, 2013.

[8] D. Hofstadter, "The Copycat Project," MIT AI Lab, Tech. Rep., Jan. 1984.

[9] M. Mitchell, *Analogy-Making as Perception: A Computer Model*. Cambridge, MA: The MIT Press, 1993.

[10] M. Mitchell and D. R. Hofstadter, "The emergence of understanding in a computer model of concepts and analogy-making," *Physica D: Nonlinear Phenomena*, vol. 42, no. 1–3, pp. 322–334, 1990.

[11] D. J. Chalmers, R. M. French, and D. R. Hofstadter, "High-level perception, representation, and analogy: A critique of artificial intelligence methodology," *Journal of Experimental & Theoretical Artificial Intelligence*, vol. 4, no. 3, pp. 185–211, 1992.

[12] R. M. French and D. Hofstadter, "Tabletop: An emergent, stochastic model of analogy-making," in *Proceedings of the 13th Annual Cognitive Science Society Conference*. Lawrence Erlbaum Associates, 1991.

[13] H. E. Foundalis, "PHAEACO: A cognitive architecture inspired by Bongard's problems," Ph.D. thesis, Indiana University, 2006.

[14] J. Rehling, "Letter Spirit (Part Two): Modeling creativity in a visual domain," Ph.D. thesis, Indiana University, 2001.

[15] E. P. Nichols, "MUSICAT: A computer model of musical listening and analogy-making," Ph.D. thesis, Indiana University, 2012.

[16] A. Linhares, "An active symbols theory of chess intuition," *Minds and Machines*, vol. 15, pp. 131–151, 2005.

[17] A. Linhares and P. Brum, "Understanding our understanding of strategic scenarios: What role do chunks play?" *Cognitive Science*, vol. 31, no. 6, pp. 989–1007, 2007.

[18] A. Linhares and A. E. T. A. Freitas, "Questioning Chase and Simon's (1973) 'Perception in Chess': The 'experience recognition' hypothesis," *New Ideas in Psychology*, vol. 28, no. 1, pp. 64–78, 2010.

[19] A. Linhares, A. E. T. A. Freitas, A. Mendes, and J. S. Silva, "Entanglement of perception and reasoning in the combinatorial game of chess: Differential errors of strategic reconstruction," *Cognitive Systems Research*, vol. 13, no. 1, pp. 72–86, 2012.

[20] A. Linhares and D. M. Chada, "What is the nature of the mind's pattern-recognition process?" *New Ideas in Psychology*, vol. 31, no. 2, pp. 108–121, 2013.

[21] A. Linhares, "The emergence of choice: Decision-making and strategic thinking through analogies," *Information Sciences*, vol. 259, pp. 36–56, 2014.

[22] A. Linhares, "Deep Vision: Seeing the essence of proofs through relational analogy," ARGO LABS Internal Report.

[23] A. Linhares, "Deep Vision," ARGO LABS Internal Report.

[24] R. Meštrović, "Wolstenholme's theorem: Its generalizations and extensions in the last hundred and fifty years (1862–2012)," *arXiv preprint arXiv:1111.3057*, 2011.

[25] R. J. McIntosh, "On the converse of Wolstenholme's theorem," *Acta Arithmetica*, vol. 71, no. 4, pp. 381–389, 1995.

[26] R. Meštrović, "Congruences for Wolstenholme primes," *Czechoslovak Mathematical Journal*, vol. 65, no. 1, pp. 237–253, 2015.

[27] Z.-W. Sun, "Super congruences and Euler numbers," *Science China Mathematics*, vol. 54, no. 12, pp. 2509–2535, 2011.

[28] Z.-W. Sun, "Open conjectures on congruences," *Nanjing University Journal of Mathematical Biquarterly*, vol. 36, no. 1, pp. 1–99, 2019.

[29] I. M. Gessel, "Wolstenholme revisited," *American Mathematical Monthly*, vol. 115, no. 5, pp. 458–461, 2008.

[30] C. Helou and G. Terjanian, "On Wolstenholme's theorem and its converse," *Journal of Number Theory*, vol. 128, no. 3, pp. 475–499, 2008.

[31] D. S. Thakur, "Power sums of polynomials over finite fields and applications: A survey," *Finite Fields and Their Applications*, vol. 32, pp. 171–191, 2015.

[32] J.-C. Pain, "A proof of Wolstenholme's theorem and congruence properties via an Egorychev-type integral," *arXiv preprint arXiv:2604.03000*, 2026.

[33] J. Bayless and D. Klyve, "Reciprocal sums as a knowledge metric: Theory, computation, and perfect numbers," *American Mathematical Monthly*, vol. 120, no. 9, pp. 822–831, 2013.

[34] K. Conrad, "The p-adic growth of harmonic sums," Expository note, University of Connecticut, 2020.

[35] W.-W. Qi, "Some congruences involving binomial coefficients and Fermat quotients," *arXiv preprint arXiv:2512.01385*, 2025.

[36] E. Treviño, "The congruence of Wolstenholme and generalized binomial coefficients related to Lucas sequences," *arXiv preprint arXiv:1409.8629*, 2014.

[37] K. Davis, "Proofs of power sum and binomial coefficient congruences via Pascal's identity," *American Mathematical Monthly*, vol. 115, no. 6, pp. 549–551, 2008.

[38] R. Brasca, C. Birkbeck, E. Rodriguez Boidi, A. Best, R. Van De Velde, and A. Yang, "A complete formalization of Fermat's Last Theorem for regular primes in Lean," *Annals of Formalized Mathematics*, vol. 1, 2025.

[39] P. Song, K. Yang, and A. Anandkumar, "Lean Copilot: Large language models as copilots for theorem proving in Lean," *arXiv preprint arXiv:2404.12534*, 2024.

[40] AlphaProof Team (Google DeepMind), "AlphaProof: AI system for formal mathematical reasoning at the IMO level," 2025.

[41] The Harmonic Team, "Aristotle: IMO-level automated theorem proving," *arXiv preprint arXiv:2510.01346*, 2025.